

CS356: A short guide to x86-64 assembly

I'll try to improve this guide over time. The goal is to collect the essential knowledge about x86-64 assembly and examples of very common patterns (`if` , `while` , `for` , procedure calls) in BombLab.

1. The registers

We have plenty of registers inside the CPU; they can store 8 bytes each and they are called:

- `%rax` , `%rbx` , `%rcx` , `%rdx` (the "a, b, c, d")
- `%rsi` , `%rdi` (source index, destination index)
- `%rsp` , `%rbp` (stack pointer, base pointer)
- `%r8` , `%r9` , ..., `%r15` (just numbers here)

We can also operate on the least significant 4, 2, or 1 bytes of these registers. To do that, we need to use different names:

- `%eax` , `%ebx` , ..., `%r8d` , `%r9d` , ..., `%r15d` (least significant 4 bytes)
- `%ax` , `%bx` , ..., `%r8w` , `%r9w` , ..., `%r15w` (least significant 2 bytes)
- `%al` , `%bl` , ... (but `%sil` and `%dil`), `%r9b` , ..., `%r15b` (least significant byte)

We also need to use instructions (e.g., `movx` or `addx`) where the suffix `x` (one of `q` , `l` , `w` , `b`) matches the size of the register:

		q (8 bytes)		l (4 bytes)	w (2 bytes)	b (1 byte)	
<code>%rax</code>	<code>%eax</code>				<code>%ax</code>		accumulate
<code>%rbx</code>	<code>%ebx</code>				<code>%bx</code>		base
<code>%rcx</code>	<code>%ecx</code>				<code>%cx</code>		counter
<code>%rdx</code>	<code>%edx</code>				<code>%dx</code>		data
<code>%rsi</code>	<code>%esi</code>				<code>%si</code>		source index
<code>%rdi</code>	<code>%edi</code>				<code>%di</code>		destination index
<code>%rsp</code>	<code>%esp</code>				<code>%sp</code>		stack pointer
<code>%rbp</code>	<code>%ebp</code>				<code>%bp</code>		base pointer

When we operate on just a portion of the register, the rest doesn't change (e.g., `movw %ax, %bx` moves data only into the least significant 2 bytes of `%rbx`). An important exception is that of instructions that end with the `l` suffix: they operate on the least significant 4 bytes but also set the most significant 4 bytes to 0. For example, `movl %eax, %ebx` moves the least significant 4 bytes of `%rax` into the least significant 4 bytes of `%rbx` but also sets the rest of `%rbx` to 0.

2. Data Movement and Addressing Modes

We can move data using `movb` , `movw` , `movl` , `movq` .

- To **move a constant into a register**: `movq $42,%rax` . Note that the constant has a `$` in front; it can also be specified in hex as `$0x2a` . The destination size has to match the suffix (`movq` for `%rax` , `movl` for `%eax` , `movw` for `%ax` , `movb` for `%al`).
- To **move a constant into a memory address**: `movq $42,0xFF112233` . Note that the address does *not* have the `$` in front. It is the initial address: the number of bytes modified in memory depends on the suffix (8 bytes with `movq` , 4 with `movl` , 2 with `movw` , 1 with `movb`).
- To **move data from one register to another**: `movq %rax,%rbx` . Note that both sizes match the suffix `q` (8 bytes). There are also instructions `movzbw` , `movzbl` , `movzbq` , `movzwl` , `movzwq` and `movsbw` , `movsbl` , `movsbq` , `movswl` , `movswq` , `movslq` to zero/sign extend from one size to another (this is frequent when you cast from one type to another in C, e.g., `int x = 10; long y = (long)x;` will use `movslq` to sign-extend `x` into `y`); `cvtq` is a shortcut for `movslq %eax,%rax` .
- To **move data from a register to memory**: `movq %rax,0xFF112233` .
- To **move data from memory to a register**: `movq 0xFF112233,%rax` .

(It's not possible to move data from memory to memory directly; we need to read into a register first, then save to a memory address.)

In the examples above, we used the constant `0xFF112233` to specify a memory address. Very often, it's useful to use some form of **indirect addressing**:

- `movq %rax,(%rbx)` : moves the 8 bytes of `%rax` to the memory address given by `%rbx` (beware: the register `%rbx` is not changed, the change happens in memory at the address given by `%rbx`). This addressing mode is very common when we use pointers in C:

```
trojan@cs356:~$ cat test.c
void f(int *p) {
    *p = 2;    // writes 2 at the address given by p
}
trojan@cs356:~$ gcc -Og -c test.c
trojan@cs356:~$ gdb -batch -ex 'disas f' test.o
Dump of assembler code for function f:
    0x0000000000000000 <+0>:    movl    $0x2,(%rdi)
    0x0000000000000006 <+6>:    retq
End of assembler dump.
```

- `movq %rax,4(%rbx)` : moves the 8 bytes of `%rax` to the memory address given by `%rbx + 4` . This is common when using pointers to structs in C:

```
trojan@cs356:~$ cat test.c
struct Data { char a; int b; };

void f(struct Data *p) {
    p->b = 2;    // writes 2 at the address of the struct plus offset of b
}
trojan@cs356:~$ gcc -Og -c test.c
trojan@cs356:~$ gdb -batch -ex 'disas f' test.o
Dump of assembler code for function f:
    0x0000000000000000 <+0>:    movl    $0x2,0x4(%rdi)
    0x0000000000000007 <+7>:    retq
End of assembler dump.
```

- `movq %rax,(%rbx,%rcx,4)` : moves the 8 bytes of `%rax` to the memory address given by `%rbx + 4 * %rcx` (the multiplier must be one of 1, 2, 4, 8). This is common when using arrays in C:

```
trojan@cs356:~$ cat test.c
void f(int *p, int i) {
    p[i] = 2;    // writes 2 at address  p + i * (size of each int)
}
trojan@cs356:~$ gcc -Og -c test.c
trojan@cs356:~$ gdb -batch -ex 'disas f' test.o
Dump of assembler code for function f:
    0x0000000000000000 <+0>:    movslq %esi,%rsi
    0x0000000000000003 <+3>:    movl    $0x2,(%rdi,%rsi,4)
    0x000000000000000a <+10>:    retq
End of assembler dump.
```

- `movq %rax,8(%rbx,%rcx,4)` combines all the previous, writing to `8 + %rbx + 4 * %rcx`.

3. Arithmetic Operations

Arithmetic operations can use the same addressing modes, register names, constants as data movements. The following examples show the variants operating on 8 bytes (suffix `q`), but 4-byte, 2-byte and 1-byte variants (suffixes `l`, `w`, `b`) are also available (similarly to data movement, variants that end with `l` set the most significant 4 bytes to 0).

- **Increment:** `incq %rax` is equivalent to `rax++` in C
- **Decrement:** `decq %rax` is equivalent to `rax--` in C
- **Negation:** `negq %rax` is equivalent to `rax = -rax` in C
- **Bitwise Negation:** `notq %rax` is equivalent to `rax = ~rax` in C
- **Addition:** `addq %rax,%rbx` is equivalent to `rbx += rax` in C (“add `rax` to `rbx`”)
- **Subtraction:** `subq %rax,%rbx` is equivalent to `rbx -= rax` (“subtract `rax` from `rbx`”)
- **Bitwise AND:** `andq %rax,%rbx` is equivalent to `rbx &= rax` in C
- **Bitwise OR:** `orq %rax,%rbx` is equivalent to `rbx |= rax` in C
- **Bitwise XOR:** `xorq %rax,%rbx` is equivalent to `rbx ^= rax` in C
- **Arithmetic Shift:**
 - `salq %cl,%rax` is equivalent to `rax = rax << cl` in C when `rax` is signed (this shifts in 0's from the right; a constant can be used too, e.g., `salq $2,%rax`)
 - `sarq %cl,%rax` is equivalent to `rax = rax >> cl` in C when `rax` is signed (this replicates the sign bit from the left; a constant can be used too, e.g., `sarq $2,%rax`)
- **Logical Shift:**
 - `shlq %cl,%rax` is equivalent to `rax = rax << cl` in C when `rax` is unsigned (this fills in 0's from the right; a constant can be used too, e.g., `shlq $2,%rax`)
 - `shrq %cl,%rax` is equivalent to `rax = rax >> cl` in C when `rax` is unsigned (this fills in 0's from the left; a constant can be used too, e.g., `shrq $2,%rax`)
- **Multiplication:**
 - `imulq %rax,%rbx` is equivalent to `rbx *= rax` (can be used for signed or unsigned, but keeps only the least significant 64 bits of the 128-bit result).
 - `imulq %rbx` multiplies `%rax` by `%rbx` and stores the result in `%rdx` (most-significant 64 bits) and `%rax` (least-significant 64 bits). The most-significant ones (`%rdx`) are correct only for signed multiplication. Note the implicit assumptions on the use of `%rax` for one of the inputs and `%rdx:%rax` for the output.
 - `mulq %rbx` is like `imulq %rbx` but for the most-significant bits saved in `%rdx` are correct for unsigned multiplication.
- **Division:**
 - `idivq %rbx` computes the signed division of `%rdx:%rax` (concatenation of two 64-bit registers) by `%rbx`. The quotient is stored in `%rax` and the remainder in `%rdx`. Similarly, `idivl %ebx` divides “`%edx:%eax`” by `%ebx` and stores quotient in `%eax` and remainder in `%edx`; or, `idivw %bx` divides “`%dx:%ax`” by `%bx` and stores quotient in `%ax` and remainder in `%dx`.

- Very often, we just want to divide a single 64-bit signed integer with `idivq`, but the instruction expects the 128-bit input `%rdx : %rax`; in that case, we can use the instruction `cqto` (no inputs) to replicate the sign bit of `%rax` all over `%rdx` before using `idivq`. And for `idivl`, we can use `cld` to replicate the sign bit of `%eax` into `%edx`; for `idivw`, we can use `cwtb` to replicate the sign bit of `%ax` into `%dx`.
- `divq %rbx` is the same but for *unsigned* division of `%rdx : %rax` by `%rbx`, storing quotient in `%rax` and remainder in `%rdx` (unsigned division gives a different result than `idivq`). If we only have a register as input (instead of `%rdx : %rax`), we can set `%rdx` to zero with `movq $0, %rdx` OR `xorq %rdx, %rdx`.
- **Load Effective Address:** `leaq 10(%rax,%rbx,4), %rcx` saves `10 + %rax + %rbx * 4` into `%rcx`. This is quite useful to combine simple arithmetic operations into one, or to compute a memory address and store it in a register `%rcx` for later use. Only the `leaq` / `leal` variants exist in x86-64 and they are used quite frequently by the compiler:

```
trojan@cs356:~$ cat test.c
int f(int x) {
    return 9*x + 5;
}
trojan@cs356:~$ gcc -Og -c test.c
trojan@cs356:~$ gdb -batch -ex 'disas f' test.o
Dump of assembler code for function f:
    0x0000000000000000 <+0>:    lea    0x5(%rdi,%rdi,8),%eax
    0x0000000000000004 <+4>:    retq
End of assembler dump.
```

4. The FLAGS Register

There is also another important register in the CPU, the `FLAGS` register.

This register is not modified directly using assembly instructions; instead, the CPU updates the `FLAGS` register **automatically after every operation**, to keep track of special true/false “conditions” associated with its different bits (the “flags”):

- **ZF** (zero flag): This flag is set to 1 if the last operation produced a result equal to 0. For example, if `%ax` is `0xFFFF`, then after `addw $1, %ax` we will see `ZF == 1` because `%rax` is equal to `???? ???? ???? 0000` (even when the rest of `%rax` is nonzero).
- **SF** (sign flag): This flag is set to 1 if the last operation produced a result with sign bit equal to 1. For example, if `%ax` is `0x7FFF`, then after `addw $1, %ax` we will see `SF == 1` because `%rax` is equal to `???? ???? ???? 8000` (even when the rest of `%rax` is zero).
- **OF** (overflow flag): This flag is set to 1 if the last operation produced signed overflow. In the previous example, `%ax` is `0x7FFF` (the largest 2-byte positive integer in 2’s complement); after `addw $1, %ax`, the result is `???? ???? ???? 8000` (note that the carry does not propagate out of `%ax`). Since `%ax` is now `0x8000` (negative), this is a case of signed overflow (`p+p=n`).
- **CF** (carry flag): This flag is set to 1 if the last operation produced unsigned overflow. If `%ax` is `0xFFFF` (the largest 2-byte unsigned integer), after `addw $1, %ax`, the result is `???? ???? ???? 0000` (again, note that the carry does not propagate out of `%ax`). This is an unsigned overflow because we added 1 to `%ax` (unsigned) and obtained something smaller.

The rules above apply to most arithmetic operations, but **there are some exceptions**:

- Condition flags are not modified at all by:
 - Data movement instructions (all `mov` variants)
 - Load effective address (`leaq` and `leal`)
 - Bitwise negation `notq` (and `l`, `w`, `b` variants)
- `incq` and `decq` (and their `l`, `w`, `b` variants) leave `CF` unchanged, even in case of unsigned overflow (e.g., from `0xFFFF` to `0x0000`).
- Bitwise `orq`, `andq`, `xorq` (and `l`, `w`, `b` variants), always set `OF` and `CF` to 0.

- Shift operations (`salq / shlq`, `sarq / shrq` and `l`, `w`, `b` variants), set `CF` to the last bit shifted out, while `OF` is undefined (shifts > 1) or it is modified by 1-bit shifts as: `0` for `sarq`, `MSB(input)` for `shrq`, `MSB(result)^CF` for left shifts (i.e., it's `1` if the MSB changed due to the shift).

In addition, these instructions (and their `l`, `w`, `b` variants) **update the FLAGS register without modifying any registers**:

- `cmpq %rax,%rbx` sets the flags like `subq %rax,%rbx`, but doesn't modify `%rbx`.
- `testq %rax,%rbx` sets the flags like `andq %rax,%rbx`, but doesn't modify `%rbx`.

5. Conditional and Unconditional Jumps

Why updating all these flags, all the time? To use them for conditional jumps:

- `jmp 0xFF112233` in an unconditional jump, which replaces the content of the instruction pointer register `%rip` with the address `0xFF112233`. The CPU will fetch the next instruction at this address, so this is indeed a "jump."
- `je 0xFF112233` ("jump if equal") jumps only if `ZF == 1`.
 - When the instruction before `je` is a comparison like `cmpq %rax,%rbx`, `ZF` is 1 at `je` only if the difference `%rbx - %rax` is zero, i.e., they are equal.
 - When the instruction before `je` is a same-register test like `testq %rax,%rax`, `ZF` is 1 at `je` only if `%rax` is zero (since `x & x == x`)
- `j1 0xFF112233` ("jump if lower") jumps only if `(SF == 1) ^ (OF == 1)`.
 - When the instruction before `j1` is a comparison like `cmpq %rax,%rbx`, `SF` is 1 at `j1` only if the difference `%rbx - %rax` is negative, i.e., `%rbx` is lower than `%rax`. The second part `^ (OF == 1)` of the condition accounts for signed overflow in the subtraction performed by `cmpq` (in the case, the output sign flag `SF` is wrong, so the XOR flips it).
 - When the instruction before `j1` is a same-register test like `testq %rax,%rax`, `SF` is 1 at `j1` only if `%rax` is negative (since `x & x == x`, `SF` is the sign flag of `%rax`).

You get the idea... `cmpq` or `testq` or other arithmetic instructions produce a change in the `FLAGS` register and then conditional jumps use the flags to decide whether to take the jump or not.

Similarly, there are `jle` ("jump if lower or equal"), `jg` ("jump if greater"), `jge` ("jump if greater or equal"), `jne` ("jump if not equal").

There are also the variants: `jb` ("jump if below"), `jbe` ("jump if below or equal"), `ja` ("jump if above"), `jae` ("jump if above or equal"). These account for **unsigned overflow** instead of signed overflow; GCC will emit them if your C code uses, for example, `unsigned int` instead of `int` inside an `if` guard.

The instruction `js` ("jump if signed") is equivalent to `j1` or `jb` when it follows `testq %rax,%rbx`: it means "jump if `%rax` is negative" in all these cases, because `testq` always sets the overflow flags `OF` and `CF` to 0.

But how can you figure out the meaning of these "compare + jump" combos without thinking about `ZF`, `SF`, `OF`, `CF` flags all the time? It's easy:

- `cmp x,y + jOP` jumps if `y OP x` (e.g., `cmpq %rax,%rbx + j1` jumps if `rbx < rax`).
- `test x,x + jOP` jumps if `x OP 0` (e.g., `testq %rax,%rax + j1` jumps if `rax < 0`).

6. Common Patterns

if statement

```
trojan@cs356:~$ cat test.c
int f(int x, int y) {
    if (x < y)
        return 5;
    else
        return 10;
}
trojan@cs356:~$ gcc -Og -c test.c
trojan@cs356:~$ gdb -batch -ex 'disas f' test.o
Dump of assembler code for function f:
    0x0000000000000000 <+0>:    cmp     %esi,%edi
    0x0000000000000002 <+2>:    jge     0xa <f+10>
    0x0000000000000004 <+4>:    mov     $0x5,%eax
    0x0000000000000009 <+9>:    retq
    0x000000000000000a <+10>:   mov     $0xa,%eax
    0x000000000000000f <+15>:   retq
End of assembler dump.
trojan@cs356:~$
```

Here:

- GDB is omitting some instruction suffixes (`mov` instead of `movl` , `cmp` instead of `cmpl`) for brevity; but you would need those in your `.s` file to compile it with GCC.
- `x` is saved into `%edi` , `y` is saved into `%esi` , the return value is in `%eax` .
- `cmp %esi,%edi + jge` takes the jump if `%edi >= %esi` , i.e., `x >= y` . If that's the case, we go to `f+10` , where we have `mov $0xa,%eax` and `retq` (returning 10).
- If we don't jump, we just move to the next instruction; that happens when `%edi >= %esi` , i.e., `x < y` (the reverse of the previous condition). In this case, we return 5.

for loop

```
trojan@cs356:~$ cat test.c
int f(int n) {
    int total = 0;
    for (int i=1; i <= n; i++) {
        total += i;
    }

    return total;
}
trojan@cs356:~$ gcc -Og -c test.c
trojan@cs356:~$ gdb -batch -ex 'disas f' test.o
Dump of assembler code for function f:
    0x0000000000000000 <+0>:    mov     $0x1,%edx
    0x0000000000000005 <+5>:    mov     $0x0,%eax
    0x000000000000000a <+10>:   cmp     %edi,%edx
    0x000000000000000c <+12>:   jg      0x15 <f+21>
    0x000000000000000e <+14>:   add     %edx,%eax
    0x0000000000000010 <+16>:   add     $0x1,%edx
    0x0000000000000013 <+19>:   jmp     0xa <f+10>
    0x0000000000000015 <+21>:   retq
End of assembler dump.
```

The C code is adding up all integers from 1 to `n` . But how do we figure that out from the assembly?

- As usual, the input `n` is in `%edi` , while the output is returned in `%eax` .
- `mov $0x1,%edx` stores 1 into `%edx` ; we don't know what this is at this point, but we take note.
- `mov $0x0,%eax` stores 0 in `%eax` ; sounds like the output value is initialized to 0.
- `cmp %edi,%edx + jg` jumps if `%edx` (1 at this point) is greater than `%edi` (which is `n`). If we jump, we go to `f+21` , where we simply return what's in `%rax` . If we don't jump, we execute the following instructions until `jmp 0xa` , which will take us to `f+10` , where `cmp` is; in other words, we execute the body of the loop and then we check the condition again.

- `add %edx,%eax adds %edx to %eax`: remember, `%eax` is our result while `%edx` is used in the `test %edx > edi`, which makes us jump to `f+21` and return (currently, `%edx` is still 1).
- `add $0x1,%edx adds 1 to %edx`.
- `jmp 0xa jumps to f+10` (unconditionally), where we check again whether `edx > edi`.

while loop

```
trojan@cs356:~$ cat test.c
int gcd(int a, int b) { // Euclid of Alexandria, 300 BC
    int tmp;

    while (b != 0) {
        tmp = b;
        b = a % b;
        a = tmp;
    }

    return a;
}
trojan@cs356:~$ gcc -Og -c test.c
trojan@cs356:~$ gdb -batch -ex 'disas gcd' test.o
Dump of assembler code for function gcd:
    0x0000000000000000 <+0>:    mov     %edi,%eax
    0x0000000000000002 <+2>:    test    %esi,%esi
    0x0000000000000004 <+4>:    je      0xf <gcd+15>
    0x0000000000000006 <+6>:    cltd
    0x0000000000000007 <+7>:    idiv    %esi
    0x0000000000000009 <+9>:    mov     %esi,%eax
    0x000000000000000b <+11>:   mov     %edx,%esi
    0x000000000000000d <+13>:   jmp     0x2 <gcd+2>
    0x000000000000000f <+15>:   retq
End of assembler dump.
```

Greatest common divisor, a great algorithm indeed:

- As usual, our inputs are in `%edi` (`a`) and `%esi` (`b`); return value in `%eax`.
- `mov %edi,%eax` saves `%edi` (`a`) into `%eax` (the return value).
- `test %esi,%esi + je` jumps to `gcd+15` when `%esi` (`b`) is 0; at `+15`, we just return. Otherwise, we execute the following instructions until `jmp` at `+13` takes us back to this test.
- `cltd` replicates the sign bit of `%eax` into all the bits of `%edx`; then, `idiv %esi` divides `%edx:%eax` (64 bits) by `%esi` (32 bits) storing quotient in `%eax` and remainder in `%edx`.
- `mov %esi,%eax` saves `%esi` (currently `b`) into `%eax` (where we had saved `a`).
- `mov %edx,%esi` saves `%edx` (the remainder of the previous division) into `%esi` (currently `b`).
- `jmp 0x2` takes us back to the beginning of the loop.

So, yeah, the logic of Euclid's algorithm is all there. But it seems somehow easy to say that, because we have the C code as well. If we only have the assembly, it helps to "rewrite it" in more understandable pseudocode. For example:

```
<+0>:    eax = edi

<+2>:    if esi == 0, go to +15
<+6>:    edx = replicas of sign bit of eax
<+7>:    edx = edx_eax % esi
        eax = edx_eax / esi
<+9>:    eax = esi
<+11>:   esi = edx
<+13>:   go to +2

<+15>:   return eax
```

Now, we notice the `go to` pattern, which we can eliminate (for [Dijkstra's sake](https://homepages.cwi.nl/~storm/teaching/reader/Dijkstra68.pdf) (<https://homepages.cwi.nl/~storm/teaching/reader/Dijkstra68.pdf>!)):

```
<+0>:    eax = edi

<+2>:    while (esi != 0) {
<+6>:        edx = replicas of sign bit of eax
<+7>:        edx = edx_eax % esi
            eax = edx_eax / esi
<+9>:        eax = esi
<+11>:       esi = edx
<+13>:    }

<+15>:    return eax
```

We also notice that we're only using the modulo of the division; so, we can simplify the code as:

```
<+0>:    eax = edi                // result = a

<+2>:    while (esi != 0) {        // while (b != 0)
<+7>:        edx = edx_eax % esi    // tmp = result % b
<+9>:        eax = esi             // result = b
<+11>:       esi = edx             // b = tmp
<+13>:    }

<+15>:    return eax              // return result
```

for loop over an array

```
trojan@cs356:~$ cat test.c
```

```
int f(int *a, int n) {
    int total = 0;

    for (int i = 0; i < n; i++) {
        total += a[i];
    }
}
```

```
    return total;
}
```

```
trojan@cs356:~$ gcc -Og -c test.c
```

```
trojan@cs356:~$ gdb -batch -ex 'disas f' test.o
```

Dump of assembler code for function f:

```
0x0000000000000000 <+0>:    mov     $0x0,%edx
0x0000000000000005 <+5>:    mov     $0x0,%eax
0x000000000000000a <+10>:   cmp     %esi,%edx
0x000000000000000c <+12>:   jge     0x19 <f+25>
0x000000000000000e <+14>:   movslq  %edx,%rcx
0x0000000000000011 <+17>:   add     (%rdi,%rcx,4),%eax
0x0000000000000014 <+20>:   add     $0x1,%edx
0x0000000000000017 <+23>:   jmp     0xa <f+10>
0x0000000000000019 <+25>:   retq
```

End of assembler dump.

This is just a variant of the `for` loop we've seen, but it also uses the addressing mode `(a,i,4)` to get element i (counting from 0) in an array where each element has size 4, i.e., at byte $a + i \times 4$.

- Again, `%rdi` is the first input argument (`a`), `%edi` the second (`n`), `%eax` the return value.
- `mov $0x0,%edx` stores 0 into `%edx` (Our counter? We don't know for sure.)
- `mov $0x0,%eax` stores 0 into `%eax` (Likely, our result.)
- `cmp %esi,%edx + jge` jumps to `f+25` when `edx >= esi`; at `f+25`, we just return. Otherwise, we execute the following instructions until `jmp` at `f+23` takes us back to this `cmp`.
- `movslq %edx,%rcx` extends `%edx` (our counter?) into 64 bits, saving the result in `%rcx`.
- `add (%rdi,%rcx,4),%eax` adds the 4 bytes at `%rdi + %rcx * 4` to `%eax`.

- add `$0x1,%edx` increments `%edx` by 1 (now we know it's a counter!)
- Then `jmp` goes back to the beginning, to check whether now `edx >= esi` (i.e., `i >= n`); note that **we stay in the for loop if the opposite condition is true** (`i < n`).

sscanf to parse a string

This is a BombLab  classic... most phases work like this:

```
trojan@cs356:~$ cat test.c
#include <stdio.h>

int phase(char *input) {
    int x;
    int y;
    char str[20];

    int count = sscanf(input, "%d %x %[^\\n]", &x, &y, str);

    if (count == 3) {
        printf("%d %d %s\\n", x, y, str);    // prints 42 10 test
        return 0;                          // means success
    } else {
        return 1;                          // means error/failure
    }
}

int main() { return phase("42 a test"); }
```

```
trojan@cs356:~$ gcc -Og test.c -o test
trojan@cs356:~$ gdb -batch -ex 'disas phase' test
Dump of assembler code for function phase:
0x0000000000001145 <+0>:    sub    $0x28,%rsp
0x0000000000001149 <+4>:    lea    0x18(%rsp),%rcx
0x000000000000114e <+9>:    lea    0x1c(%rsp),%rdx
0x0000000000001153 <+14>:   mov    %rsp,%r8
0x0000000000001156 <+17>:   lea    0xea7(%rip),%rsi    # 0x2004
0x000000000000115d <+24>:   mov    $0x0,%eax
0x0000000000001162 <+29>:   callq  0x1040 <__isoc99_sscanf@plt>
0x0000000000001167 <+34>:   cmp    $0x3,%eax
0x000000000000116a <+37>:   je     0x1176 <phase+49>
0x000000000000116c <+39>:   mov    $0x1,%eax
0x0000000000001171 <+44>:   add    $0x28,%rsp
0x0000000000001175 <+48>:   retq
0x0000000000001176 <+49>:   mov    %rsp,%rcx
0x0000000000001179 <+52>:   mov    0x18(%rsp),%edx
0x000000000000117d <+56>:   mov    0x1c(%rsp),%esi
0x0000000000001181 <+60>:   lea    0xe88(%rip),%rdi    # 0x2010
0x0000000000001188 <+67>:   mov    $0x0,%eax
0x000000000000118d <+72>:   callq  0x1030 <printf@plt>
0x0000000000001192 <+77>:   mov    $0x0,%eax
0x0000000000001197 <+82>:   jmp    0x1171 <phase+44>
End of assembler dump.
```

First of all, what is this C program doing?

- The function `phase` receives an `input` string (an array of `char`, with `0x00` to mark the end of the string); in assembly, `%rdi` will contain the address of the first character of the string.
- It allocates three local variables: `x` (4 bytes), `y` (4 bytes), `str` (20 bytes). These variables are allocated in memory, on the stack, not in some register.
- It calls `sscanf` passing: a pointer to the beginning of the input string to parse (`%rdi`); a pointer to the format string `%d %x %[^\\n]` (`%rsi`), meaning that an integer in decimal format, one in hex format and a string (without newlines) should be parsed; the addresses `&x`, `&y`, `str` where the parsed values should be saved (`%rdx`, `%rcx`, `r8`).
- It checks the return value of `sscanf`: if all three arguments are parsed correctly, the return argument is going to be 3 and the phase returns 0 (no error); otherwise, it returns 1 (error).

Now, let's look at the assembly, step-by-step:

- `sub $0x28,%rsp` decreases the stack pointer `%rsp` by 40 to make space for local variables.
- `lea 0x18(%rsp),%rcx` saves the address of `y` into `%rcx` (the choice of where to place `y` in the allocated space is arbitrary).
- `lea 0x1c(%rsp),%rdx` saves the address of `x` into `%rdx`.
- `mov %rsp,%r8` saves the address of `str` into `%r8`.
- `lea 0xea7(%rip),%rsi` stores the address of the format string (a constant relative to `%rip`) into `%rsi`.
- `mov $0x0,%eax / callq 0x1040 <__isoc99_sscanf@plt>` saves 0 into `%rax` and calls `sscanf`.
- `cmp $0x3,%eax + je` jumps to +49 if `sscanf` has returned 3 into `%rax`; otherwise, continues to the next instruction.
- `mov $0x1,%eax` saves 1 into `%eax` (the return value).
- `<+44>: add $0x28,%rsp` increases the stack pointer by `0x28` to deallocate the memory that was initially allocated.
- `retq` returns.
- `<+49>: mov %rsp,%rcx` : we get here if the number of parsed arguments was right; this instruction saves the address of `str` into `%rcx` (4th argument to `printf` later).
- `mov 0x18(%rsp),%edx` saves the value of `y` into `%edx` (3rd argument to `printf`).
- `mov 0x1c(%rsp),%esi` saves the value of `x` into `%rsi` (2nd argument to `printf`).
- `lea 0xe88(%rip),%rdi` saves the address of the format string into `%rdi` (1st argument to `printf`).
- `mov $0x0,%eax / callq 0x1030 <printf@plt>` saves 0 into `%rax` and calls `printf`.
- `mov $0x0,%eax` saves 0 into `%rax` (our return value, no error).
- `jmp 0x1171 <phase+44>` jumps to +44, where it deallocates memory from the stack and returns.